

Real-Time Temperature Monitoring: An Arduino-MATLAB Integration

EE427/527 : Industrial Controls

Charith Sai Sathvik Ale

UAB

Contents

- Introduction
- Objectives of the Project
- Technical Stack Overview
- Hardware Setup
- Software Configuration
- Data Flow Process
- Demonstration
- Challenges and Solutions
- Future Enhancements
- Conclusion
- References

Introduction

Purpose: Develop a real-time temperature monitoring system integrating Arduino hardware with MATLAB software.

Significance: Addresses the need for precise, efficient, and scalable temperature monitoring in various applications, including environmental control, industrial processes, and safety systems.

Technology Integration: Combines the versatility of Arduino microcontrollers with the powerful data analysis and visualization capabilities of MATLAB.

Advantages:

- Enhanced accuracy and real-time data processing.
- Cost-effective solution adaptable to both simple and complex needs.
- Improved data accessibility and visualization for quicker decision-making.
- **Application Scope:** Ranges from educational uses to advanced industrial systems, demonstrating broad utility and adaptability.

Enhancing Environmental Sensing with Arduino-MATLAB Integration

Innovative Monitoring Approach:

- Integrates Arduino microcontrollers with MATLAB for a networked, real-time monitoring system.
- Facilitates dense data collection from localized sensors, enhancing detail and accuracy.

Real-time Data and Enhanced Accessibility:

- Provides immediate temperature data updates, accessible remotely via MATLAB interface.
- Enables prompt adjustments and responses to temperature fluctuations and anomalies.

Cost-effectiveness and Scalability:

- Offers a budget-friendly solution for both educational purposes and industrial applications.
- Easily scalable from small setups to larger, more complex systems without significant cost increases.

Data Integration and Intelligent Analysis:

- Combines real-time sensor data with advanced MATLAB data processing and visualization tools.
- Utilizes MATLAB's analytical capabilities to perform sophisticated temperature trend analysis and predictions.

Technical Stack Overview

Arduino:

- Arduino is an open-source electronics platform based on easy-to-use hardware and software.
- It consists of a microcontroller that can be programmed to read inputs from sensors and control outputs like lights, motors, and displays.
- Arduino boards are widely used for prototyping and building interactive electronic projects due to their simplicity and versatility.

Open-PLC:

- Open-PLC (Open Programmable Logic Controller) is an open-source platform used for industrial automation.
- It allows users to create and execute programs that control industrial processes, machinery, and equipment.
- Open-PLC can run on various hardware platforms, including Arduino, Raspberry Pi, and industrial PCs, making it flexible and adaptable to different applications.

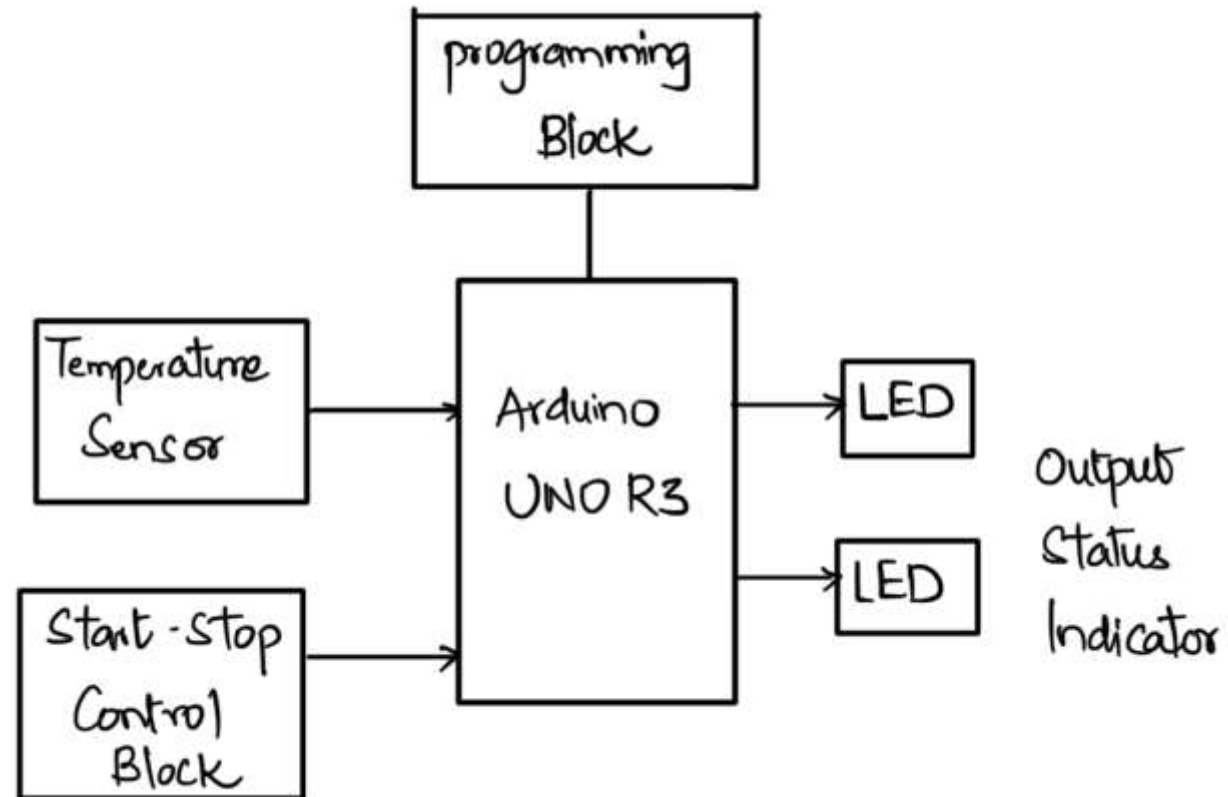
MATLAB:

- MATLAB is a high-performance language for technical computing developed by MathWorks.
- It integrates computation, visualization, and programming in an easy-to-use environment where problems and solutions are expressed in familiar mathematical notation.
- In the context of this technical stack, MATLAB is utilized for creating plots and visualizations, like the one displayed, to analyze and present data in a clear and impactful way.
- Its powerful toolboxes and functions make it an indispensable resource for data analysis, algorithm development, and various other engineering and scientific applications.

System Architecture

Block Diagram

- The system architecture is depicted in the block diagram below, illustrating the interconnected components and their functions in the environmental monitoring system.



Hardware Setup

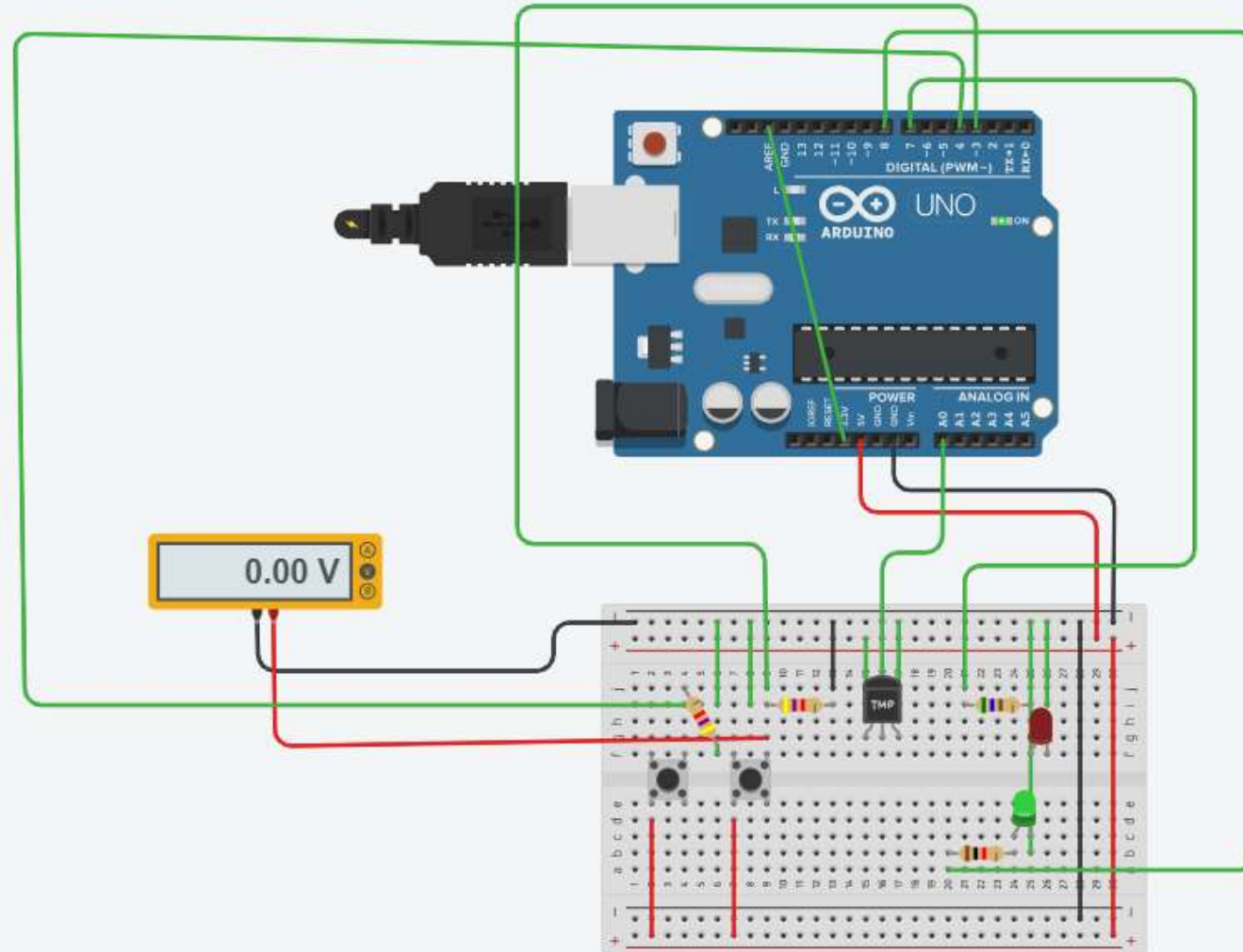
Details about the Arduino Board and Temperature Sensor Used:

Arduino Board: The hardware setup utilizes the Arduino microcontroller board, specifically the Arduino Uno, due to its simplicity, affordability, and wide availability. The Arduino Uno features digital and analog input/output pins, making it suitable for interfacing with various sensors and actuators.

Temperature Sensor: For temperature monitoring, the system employs the TMP35 digital temperature sensor. The TMP35 is a versatile and accurate sensor that provides temperature readings with a resolution of up to 12 bits. It uses the OneWire protocol for communication, allowing multiple sensors to be connected to a single digital pin on the Arduino board.

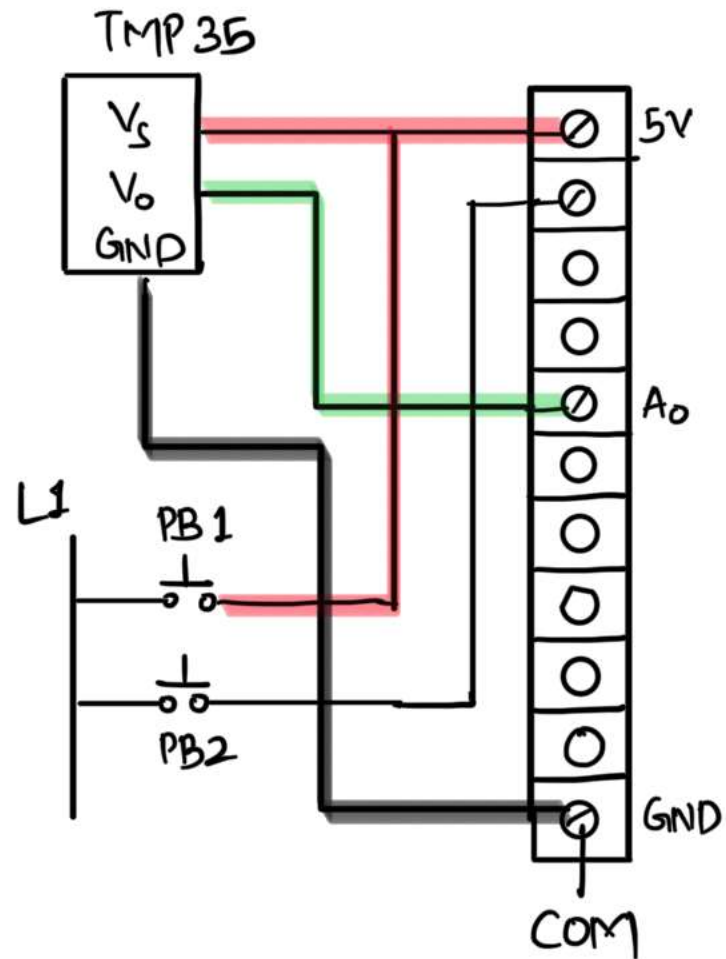
Wiring Diagram:

The wiring diagram below illustrates the connections between the Arduino board and the TMP35 temperature sensor.

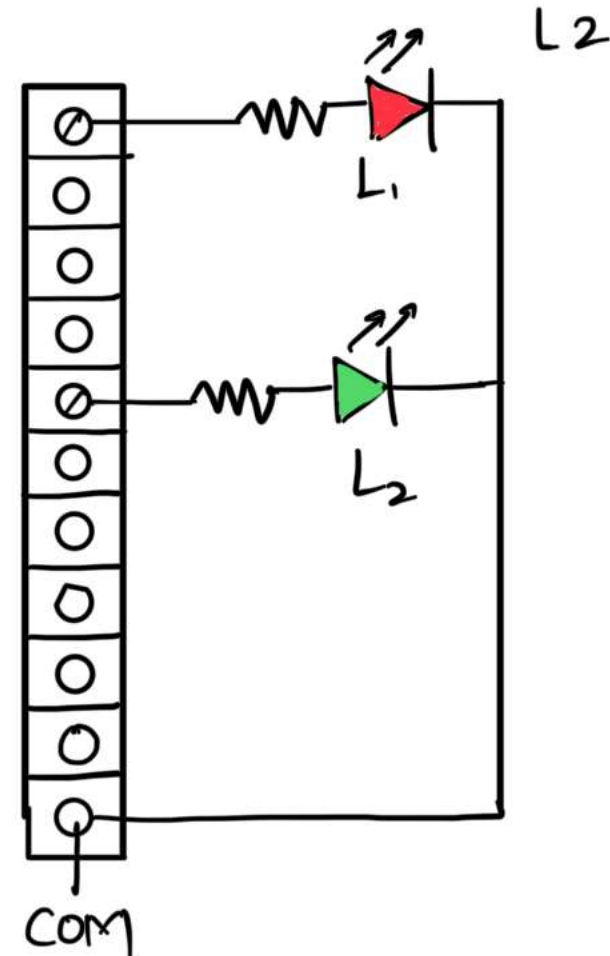


Input Output Module Wiring Diagram

Input Module



Output Module



Arduino Connections: The TMP35 sensor is connected to the Arduino board using three wires: VCC (power), GND (ground), and Vout (output). The DATA pin is connected to a digital pin on the Arduino for data transmission.

- The choice of Arduino Uno for the hardware setup is based on its ease of use, compatibility with a wide range of sensors and actuators, and robust community support.
- The TMP35 temperature sensor is selected for its accuracy, digital interface, and simplicity of integration with the Arduino platform.
- The wiring diagram illustrates the connections required to interface the TMP35 sensor with the Arduino board. It shows how the sensor's power, ground, and data pins are connected to the corresponding pins on the Arduino.
- With this hardware setup, the system is capable of accurately measuring temperature readings and transmitting the data to the Arduino for further processing and analysis.

Software Configuration

Steps to Set Up Open-PLC on Arduino:

1. **Install Open-PLC Runtime:** Begin by downloading and installing the Open-PLC runtime on your computer. The Open-PLC runtime is available for Windows, Linux, and macOS platforms.
 2. **Upload Firmware to Arduino:** Connect the Arduino board to your computer via USB and upload the Open-PLC firmware to the Arduino using the Arduino IDE or Open-PLC Editor. The firmware enables the Arduino to act as a programmable logic controller (PLC) capable of executing ladder logic programs.
 3. **Configure Open-PLC Editor:** Open the Open-PLC Editor and create a new project. Define the inputs, outputs, and ladder logic program to control the environmental parameters based on sensor data received from the Arduino.
 4. **Uploading Project to Arduino:** Once the project is configured, upload it to the Arduino board using the Open-PLC Editor. The Arduino will now execute the ladder logic program and control the environmental parameters accordingly.
- Setting up Open-PLC on the Arduino involves installing the Open-PLC runtime, uploading firmware to the Arduino, configuring the Open-PLC Editor, and uploading the project to the Arduino board.

Data Collection Configuration: Program Arduino to monitor and record converter metrics. Important parameters: Input/Output voltage, Current.

Integration with Arduino: Implement MATLAB Support Package for Arduino. Enables direct data acquisition from Arduino into MATLAB.

MATLAB Scripting: Write MATLAB scripts to read and process data from Arduino. Include serial communication initiation and data processing.

Data Organization: Structure data in MATLAB for analysis. Use appropriate data structures like matrices or tables.

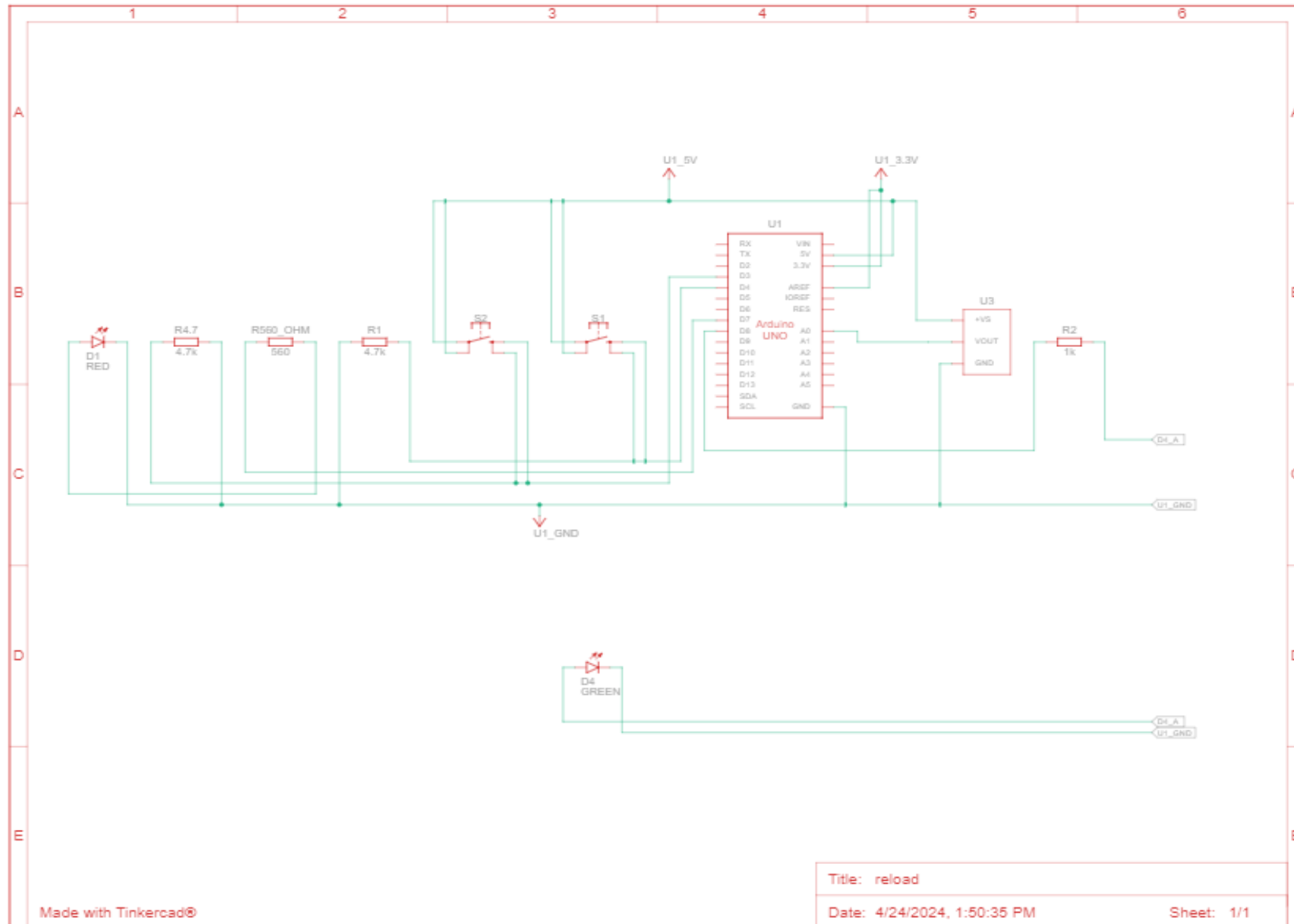
Data Visualization: Use MATLAB's plotting tools to visualize data. Create dynamic plots for real-time visualization of converter performance.

Analysis and Interpretation: Analyze converter efficiency and performance directly within MATLAB. Utilize statistical and analytical tools available in MATLAB.

Sharing Insights: Export graphs and create interactive apps with MATLAB App Designer. Share findings and visualizations with team members or stakeholders.

- Customize MATLAB code to match specific project requirements.
- Ensure real-time data is accurately captured for reliable analysis.

SCHEMATIC DIAGRAM



Arduino C++ code for Start- Stop Control -with comments

```
#define LED_PIN2 8

#define LED_PIN 7

#define SWITCH_PIN_ON 3 // Pin for the button to turn the LED on

#define SWITCH_PIN_OFF 4 // Pin for the button to turn the LED off

#define aref_voltage 3.3

int tempPin=0;

int tempReading;

void setup() {

    pinMode(LED_PIN, OUTPUT);

    pinMode(LED_PIN2, OUTPUT);

    pinMode(SWITCH_PIN_ON, INPUT);

    pinMode(SWITCH_PIN_OFF, INPUT);

    Serial.begin(9600);

    analogReference(EXTERNAL);

}

void loop() {

    static bool ledState = false;

    static bool ledState1 = false;

    // Variable to store the state of the LED
```

```
if (digitalRead(SWITCH_PIN_ON) == HIGH) {  
    ledState = true; // Set the LED to be permanently on  
}  
  
if (digitalRead(SWITCH_PIN_OFF) == HIGH) {  
    ledState = false; // Set the LED to be permanently off  
}  
  
if(ledState==true){  
    tempReading = analogRead(tempPin);  
    Serial.print("RAW Input=");  
    Serial.print(tempReading);  
  
    float voltage = tempReading * aref_voltage;  
    voltage /= 1024.0;  
    Serial.print("-");  
    Serial.print(voltage); Serial.print("volts");  
  
    float temperatureC = (voltage - 0.5) * 100;  
    Serial.print("-Temperature");  
    Serial.print(temperatureC); Serial.println("degrees C");  
    if(temperatureC>=23){  
        ledState1=true;  
  
    }  
}
```

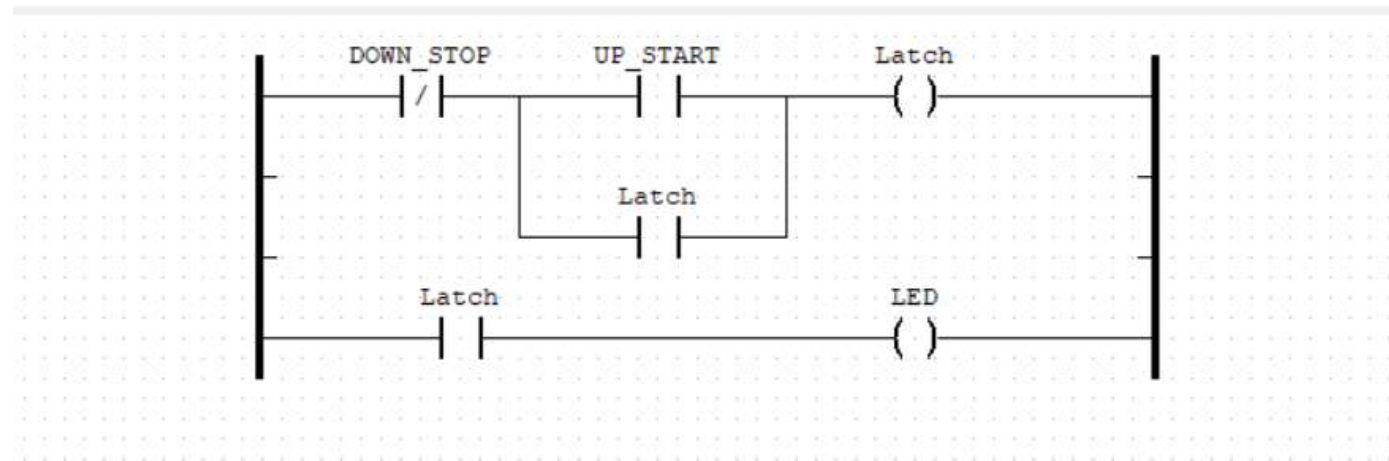
```
else
    {ledState1=false;}

    delay(1000);
    digitalWrite(LED_PIN2, ledState1 ? HIGH : LOW);
}

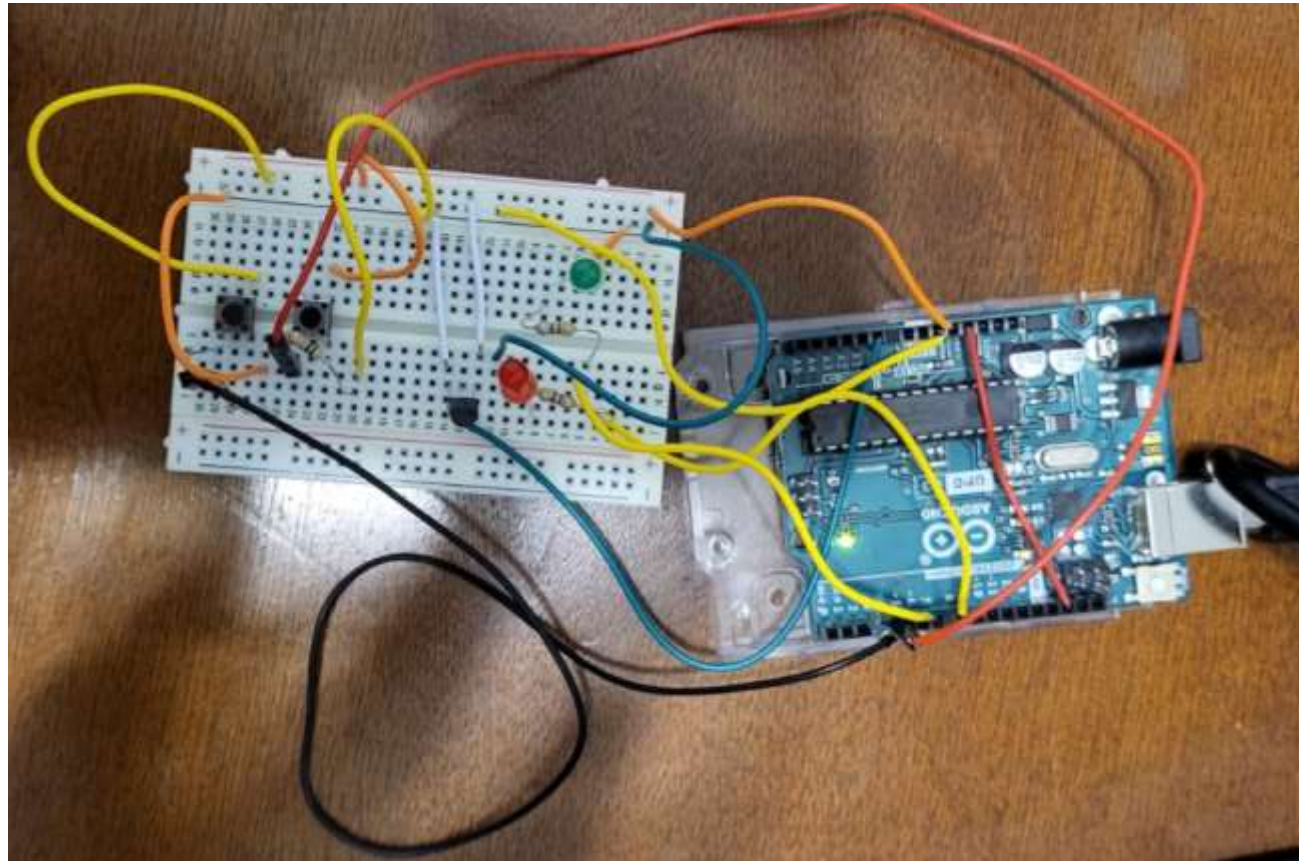
digitalWrite(LED_PIN2, ledState1 ? HIGH : LOW);
digitalWrite(LED_PIN, ledState ? HIGH : LOW);
// Update the LED based on the ledState
}
```


Ladder Diagram with Tag Listing Table

Description:		Class Filter: All ▼				
#	Name	Class	Type	Location	Initial Value	Option
1	UP_START	Local	BOOL	%IX0.1		
2	DOWN_STOP	Local	BOOL	%IX0.2		
3	Latch	Local	BOOL			
4	LED	Local	BOOL	%QX0.0		



Picture of wired Arduino Uno – Open-PLC Controller prototype.



MATLAB Code Snippet

```
clear all
r=arduino('com7','uno')
v=zeros(500,1);
t0=datetime('now');
for i=1:500
v(i)=readVoltage(r,'A0');
t(i)=datetime('now')-t0;endtep=(v-0.5)*100;
```

Code Description

1.Arduino Connection

This line establishes a connection to an Arduino Uno on COM port 7.

2. Variable Initialization:

- ``v`` is initialized as a zero vector with 500 elements to store voltage readings.
- ``t0`` captures the current date and time to use as a reference point for timing each measurement.

3. Data Collection Loop:

- The loop reads the voltage from pin A0 of the Arduino 500 times.
- Voltage readings are stored in ``v``.
- Time differences (``t``) from the start (``t0``) are calculated for each reading, but note there is an issue here—``t`` is not preinitialized, which can cause errors or inefficient code execution in MATLAB.

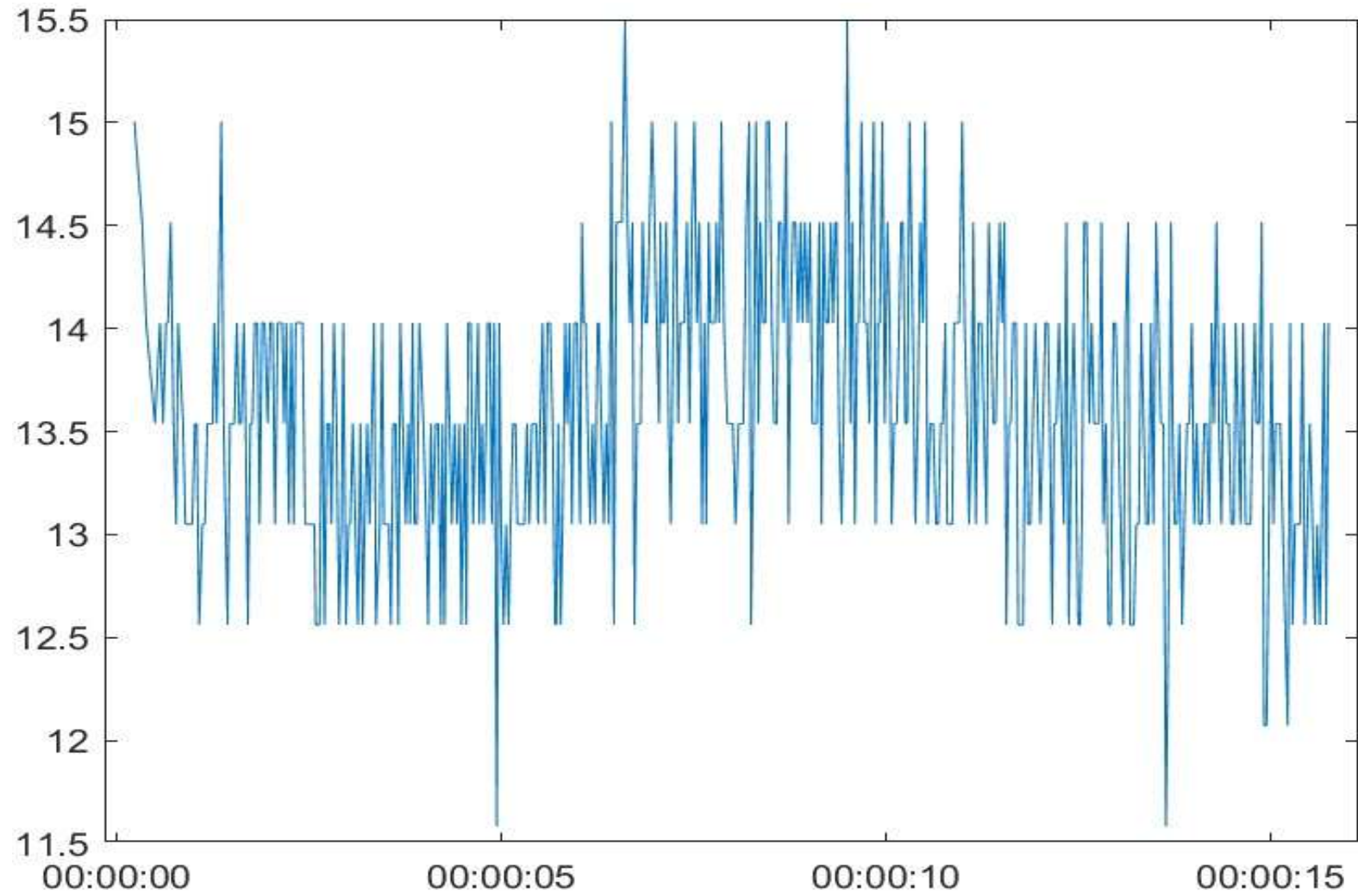
4. Data Processing:

- This line converts the voltage readings into another form of measurement, likely temperature in Celsius, assuming a linear transformation from voltage to temperature (e.g., output from a temperature sensor with a voltage offset and scale factor).

5. Graph Description:

- The x-axis of the graph would represent time (``t``), indicating the duration since the start of measurements.
- The y-axis would display the processed temperature data (``temp``).
- Since ``v`` varies, ``temp`` would show fluctuations corresponding to changes in the measured voltage over time, transformed to temperature scale.
- The graph should show a line plot of temperature vs. time, illustrating how the temperature changes during the measurement period.

Visualization Graph



Challenges and Solutions

Identifying Challenges Faced During the Project:

1. **Hardware Compatibility Issues:** One of the initial challenges encountered was ensuring compatibility between the Arduino board and the sensors. Different sensor models may require specific libraries or configurations, leading to compatibility issues.
2. **Data Transmission Reliability:** Another challenge was ensuring reliable data transmission from the Arduino to the data visualization platform. Factors such as network connectivity, data packet loss, and communication protocols could affect the reliability of data transmission.
3. **Integration Complexity:** Integrating multiple components, such as Arduino, Open-PLC, MATLAB, into a cohesive system posed a significant challenge. Each component may have its configuration requirements and compatibility constraints.

Proposing Solutions to Overcome Challenges:

1. **Research and Testing:** Extensive research and testing were conducted to identify compatible sensor models and libraries for the Arduino platform. Prototyping and experimentation helped validate hardware configurations before full-scale deployment.
2. **Error Handling and Redundancy:** To address data transmission issues, error handling mechanisms and redundancy measures were implemented. This included implementing data validation algorithms, packet retransmission strategies, and backup data storage solutions.
3. **Modular Design Approach:** A modular design approach was adopted to simplify integration complexity. Each component of the system was developed as a separate module with well-defined interfaces, allowing for easier integration and troubleshooting.

Implementing Solutions and Lessons Learned:

1. Iterative Development: Solutions were implemented iteratively, with frequent testing and feedback loops to refine and improve system performance. This iterative approach allowed for continuous improvement and optimization of the environmental monitoring system.

2. Collaboration and Knowledge Sharing: Collaboration among team members and knowledge sharing within the project team were essential for overcoming challenges effectively. Regular meetings, brainstorming sessions, and documentation helped streamline problem-solving efforts and fostered a culture of innovation.

- The project faced several challenges during the implementation phase, ranging from hardware compatibility issues to data transmission reliability concerns.
- Through thorough research, testing, and collaboration, solutions were proposed and implemented to address these challenges effectively.
- The iterative development approach allowed for continuous improvement and optimization of the environmental monitoring system, leading to a robust and reliable solution.
- By sharing lessons learned and best practices, the project team was able to overcome challenges and achieve success in delivering a functional and impactful environmental monitoring solution.

Future Enhancements

Identifying Potential Improvements and Additions to the System:

1. **Enhanced Sensor Capabilities:** Upgrading to advanced sensor models with higher accuracy and additional environmental parameters could provide more comprehensive monitoring capabilities. For example, integrating gas sensors for detecting pollutants or particulate matter sensors for air quality monitoring.
2. **Remote Monitoring and Control:** Implementing remote monitoring and control features would enable users to access and manage the environmental monitoring system from anywhere via the internet. This could include remote data visualization, real-time alerts, and remote device management.
3. **Predictive Analytics:** Incorporating predictive analytics algorithms could enable the system to forecast environmental trends and anticipate potential issues before they occur. By analyzing historical data and environmental patterns, predictive models could provide valuable insights for proactive decision-making.
4. **Integration with Smart Devices:** Integrating the environmental monitoring system with smart devices and IoT platforms could enhance automation and interoperability. For example, integrating with smart thermostats or HVAC systems to adjust indoor temperature based on environmental conditions.

Proposing Strategies for Implementation:

1. **Research and Development:** Conducting further research and development to explore advanced sensor technologies and predictive analytics algorithms. Collaboration with academic institutions or industry partners could facilitate access to cutting-edge research and expertise.
2. **Prototype Testing:** Developing prototypes and conducting pilot studies to evaluate the feasibility and effectiveness of proposed enhancements. User feedback and field testing would provide valuable insights for refining and optimizing the system.
3. **Scalability and Flexibility:** Designing the system architecture with scalability and flexibility in mind to accommodate future enhancements seamlessly. Modular design principles and open-source technologies could facilitate easy integration and expansion of new features.

Benefits of Future Enhancements:

1. **Enhanced Monitoring Accuracy:** Advanced sensor capabilities and predictive analytics algorithms would improve the accuracy and reliability of environmental monitoring data, enabling more informed decision-making.
2. **Increased Efficiency and Automation:** Remote monitoring and control features would streamline operations and reduce manual intervention, leading to increased efficiency and cost savings.
3. **Proactive Environmental Management:** Predictive analytics capabilities would enable proactive environmental management by identifying trends and potential issues before they escalate, ultimately contributing to environmental sustainability.

- While the current environmental monitoring system is functional, there are several opportunities for future enhancements and improvements.
- By investing in research and development, prototype testing, and collaboration, the system can evolve to meet the evolving needs and challenges of environmental monitoring.
- Future enhancements such as advanced sensor capabilities, remote monitoring and control, predictive analytics, and integration with smart devices would unlock new possibilities for environmental management and sustainability.

Conclusion

Recap of What the Project Achieved:

1. **Functional Environmental Monitoring System:** The project successfully developed and implemented a functional environmental monitoring system using Arduino, Open-PLC, MATLAB.
2. **Real-Time Data Acquisition and Visualization:** The system enables real-time data acquisition from environmental sensors and visualization of sensor data through web-based dashboards.
3. **Control Logic Implementation:** Control logic was implemented to regulate environmental parameters based on sensor data, enhancing the system's utility and effectiveness.

Final Thoughts on the Impact of the Project:

1. **Empowering Environmental Management:** The project's success empowers environmental management by providing stakeholders with valuable insights into environmental conditions and trends.
2. **Enhancing Decision-Making:** The availability of real-time environmental data enables informed decision-making and proactive intervention to mitigate environmental risks and promote sustainability.
3. **Driving Innovation and Collaboration:** The project serves as a catalyst for innovation and collaboration, inspiring future research and development efforts in the field of environmental monitoring and IoT technology.

CONCLUSION

- In conclusion, the project has achieved its objective of developing an effective environmental monitoring system, providing stakeholders with valuable insights into environmental conditions and trends.
- By enabling real-time data acquisition and visualization, the system enhances decision-making and empowers proactive environmental management.
- Moving forward, the project's impact extends beyond its immediate scope, driving innovation and collaboration in the field of environmental monitoring and IoT technology.
- As we reflect on the journey and accomplishments of this project, we are reminded of the importance of leveraging technology for the greater good and making a positive impact on our environment and society.

REFERENCES

Sources of Information:

1. "Arduino Uno Documentation" - Arduino Official Website
2. "Open-PLC Documentation" – Open-PLC Project Website
3. “MATLAB Documentation”

Libraries and Tools Used:

1. Arduino IDE - Integrated Development Environment for Arduino Programming
2. Open-PLC Editor - Software for Programming Open-PLC Controllers
3. MATLAB - Platform for Collecting, Analyzing, and Visualizing Data

Additional Resources:

1. Online tutorials and forums: Arduino Forum, Stack Overflow, GitHub repositories.
2. Academic papers and research articles related to environmental monitoring and IoT technology.

The successful completion of this project was made possible by leveraging various sources of information, libraries, and tools.

- The Arduino Uno documentation provided essential guidance on programming the Arduino board and interfacing with sensors.
- Open-PLC documentation and tools facilitated the development of ladder logic programs for control logic implementation.
- MATLAB documentation guided the creation of flow-based applications for data visualization and dashboard development.
- In addition to official documentation, online tutorials, forums, and academic resources played a significant role in overcoming challenges and finding solutions throughout the project.
- By acknowledging these sources of information and tools used, we ensure transparency and give credit to the individuals and organizations that contributed to the success of this project.